

## Final Report

### Comprehensive OOP Project — CSE 1116: Object Oriented Programming Laboratory

|               |                                                                                                    |                 |                                 |
|---------------|----------------------------------------------------------------------------------------------------|-----------------|---------------------------------|
| Course Title  | Object Oriented Programming Laboratory                                                             |                 |                                 |
| Project Title | Smart Food Delivery Rider Assignment & Dispatch System: An OOP-Driven Real-Time Logistics Solution |                 |                                 |
| Course Code   | CSE 1116                                                                                           | Instructor      | Course Teacher / Lab Instructor |
| Student Name  | Pranta Mohajan                                                                                     | Student ID      | 0222510005101025                |
| Team Members  | Shreya Sen (0222510005101029)<br>Arpita Paul (0222510005101033)                                    |                 |                                 |
| Section       | Section A / Session 2026                                                                           | Submission Date | September 29, 2026              |

## 1. Scenario Description

In high-density urban areas such as Chattogram, on-demand food delivery platforms experience significant logistical bottlenecks during rush hours. Traditional dispatch procedures rely on manual guesswork, leading to severe vehicle-trip mismatches: deploying motorized motorbikes for short trips ( $\leq 5$  km) incurs excessive fuel consumption and carbon emissions, whereas assigning bicycles to long trips ( $> 5$  km) results in cold food, delayed deliveries, and rider fatigue. Furthermore, peak-hour fleet saturation often crashes naive dispatch software. To resolve these challenges, this project engineers the **Smart Food Delivery Rider Assignment & Dispatch System** in Java. The system automates vehicle selection via distance-aware routing logic, enforces an eco-routing paradigm ( $\leq 5$  km allocated to eco-friendly cycle couriers;  $> 5$  km to motorized bikes), calculates travel times dynamically through method overriding, safeguards the fleet lifecycle using a custom checked exception (`NoRiderAvailableException`), and provides an interactive Java Swing dashboard equipped with real-time fleet tables, automated pending queue processing, and atomic order deletion.

## 2. Class Structure

**Relevant class(es):** Person, Assignable, Rider, BikeRider, CycleRider, Order, NoRiderAvailableException, DeliveryManager, RiderAssignmentGUI

The system is organized into modular model, controller, exception, and graphical presentation layers across nine core classes and interfaces:

- **Person (Abstract Class):** Root base class encapsulating foundational human identity data (name, id, phone) and declaring the abstract method `public abstract String getRoleDescription()`.
- **Assignable (Interface):** Defines the operational dispatch contract containing `void assignOrder(Order o)` and `void completeOrder()`, decoupling rider implementations from controller dispatch mechanics.
- **Rider (Abstract Class):** Extends `Person` and implements `Assignable`. Encapsulates operational fields (`isAvailable`, `vehicleType`, `currentOrder`) and declares the polymorphic abstraction `public abstract double calculateDeliveryTime(double distanceKm)`.
- **BikeRider (Concrete Class):** Subclass extending `Rider`. Models motorized vehicles, adding attributes `fuelLevel` and `maxDistanceKm`, and overriding `calculateDeliveryTime()` based on an average speed of 30 km/h.
- **CycleRider (Concrete Class):** Subclass extending `Rider`. Models human-powered bicycles, adding `maxLoadKg`, and overriding `calculateDeliveryTime()` based on an average urban speed of 12 km/h.
- **Order (Model Class):** Encapsulates parcel and transaction metadata including `orderId`, `customerName`, `restaurant`, `deliveryAddress`, and `status` (e.g., *Pending*, *Assigned*, *Delivered*).
- **NoRiderAvailableException (Custom Checked Exception):** Inherits from `java.lang.Exception`. Thrown when the delivery fleet is fully saturated or an ineligible vehicle preference is requested.

- **DeliveryManager (Controller Class):** Acts as the logistical engine, encapsulating collections (`HashMap<String, Rider>` and `ArrayList<Order>`), method overloading for rider search, distance-tiered assignment logic, and pending order reassignment.
- **RiderAssignmentGUI (View / Presentation Class):** Inherits from `JFrame`. Implements the dual-panel desktop dashboard featuring registration forms, dispatch triggers, synchronized table views, and atomic order removal.

3. Attributes

Relevant class(es): Person, Rider, BikeRider, CycleRider, Order, DeliveryManager

| Class           | Attribute Name                                     | Data Type              | Access Modifier | Design Justification                                                                              |
|-----------------|----------------------------------------------------|------------------------|-----------------|---------------------------------------------------------------------------------------------------|
| Person          | name, id, phone                                    | String                 | private         | Encapsulates personal identity and unique primary keys, preventing unauthorized outside mutation. |
| Rider           | isAvailable                                        | boolean                | private         | Tracks operational readiness ( <code>true</code> = Available, <code>false</code> = On Delivery).  |
| Rider           | vehicleType                                        | String                 | private         | Specifies vehicle category ("Bike" or "Bicycle") for dispatch filtering.                          |
| Rider           | currentOrder                                       | Order                  | private         | Maintains an aggregation reference to the active parcel currently in transit.                     |
| BikeRider       | fuelLevel                                          | double                 | private         | Tracks remaining fuel percentage (0.0% to 100.0%) essential for combustion engine trips.          |
| BikeRider       | maxDistanceKm                                      | double                 | private         | Calculated maximum travel perimeter based on fuel volume ( <code>Fuel × 0.5 km</code> ).          |
| CycleRider      | maxLoadKg                                          | double                 | private         | Physical payload carrying limit in kilograms to prevent courier physical strain.                  |
| Order           | orderId, customerName, restaurant, deliveryAddress | String                 | private         | Core logistical transaction metadata required for pickup, tracking, and customer drop-off.        |
| Order           | status                                             | String                 | private         | Lifecycle state indicator ("Pending", "Assigned", "Delivered").                                   |
| DeliveryManager | riderMap                                           | HashMap<String, Rider> | private         | High-efficiency key-value associative collection providing O(1) rider lookup and updates.         |
| DeliveryManager | orderList                                          | ArrayList<Order>       | private         | Dynamic array maintaining chronological placement order and audit tracking history.               |

4. Constructors

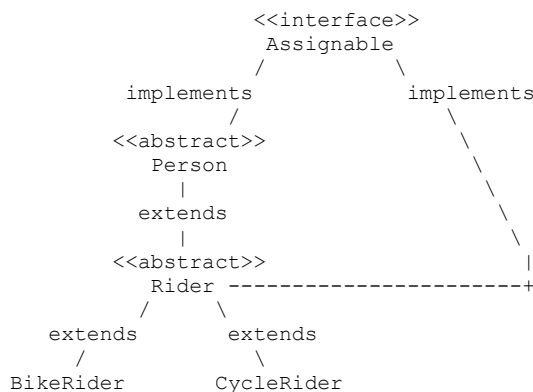
Relevant class(es)/constructor(s): Person, Rider, BikeRider, CycleRider, Order, DeliveryManager constructors

- **Person(String name, String id, String phone):** Base constructor guaranteeing that human identity fields are fully initialized and never left in an invalid state.
- **Rider(String name, String id, String phone, String vehicleType):** Chains to `super(name, id, phone)`, autonomously initializes `this.isAvailable = true` and `this.currentOrder = null`, making new couriers immediately ready for dispatch.
- **BikeRider(String name, String id, String phone, double fuelLevel, double maxDistanceKm):** Explicitly invokes `super(name, id, phone, "Bike")` to eliminate vehicle string typos while binding fuel and distance attributes.

- **CycleRider(String name, String id, String phone, double maxLoadKg):** Invokes `super(name, id, phone, "Bicycle")` while initializing payload limits.
- **Order(String orderId, String customerName, String restaurant, String deliveryAddress):** Sets origin and destination coordinates while autonomously defaulting initial status to "Pending".
- **DeliveryManager():** Default constructor allocating memory for underlying collection instances (`new HashMap<>()` and `new ArrayList<>()`) to guarantee null-safety during client invocations.

## 5. Inheritance Hierarchy

**Relevant class(es)/interface:** Assignable, Person, Rider, BikeRider, CycleRider



### Justification of Hierarchy:

- **Grandparent (Person):** Declared abstract to prevent meaningless instantiation of an unidentified human entity. Centralizes universal human attributes.
- **Parent (Rider):** Bridges real-world identity with logistics operations, isolating delivery state flags from base human characteristics.
- **Children (BikeRider & CycleRider):** Separates mechanical fuel/range constraints from physiological physical payload limitations.
- **Interface (Assignable):** Enforces behavioral consistency. Guarantees that any delivery courier implements `assignOrder()` and `completeOrder()` uniformly.

## 6. Polymorphism

**Relevant class(es)/method(s):** `DeliveryManager.searchRider()`, `Rider.calculateDeliveryTime()`, `getRoleDescription()`

- **1. Compile-Time Polymorphism (Method Overloading):** Implemented in `DeliveryManager` through two overloaded query signatures:
  - `public Rider searchRider(String id):` Instantaneous  $O(1)$  search by unique ID key.
  - `public Rider searchRider(String vehicleType, boolean onlyAvailable):` Fleet filter iterating over vehicle types and active availability flags.
- **2. Runtime Polymorphism (Method Overriding & Dynamic Binding):** The abstract method `calculateDeliveryTime(double distanceKm)` declared in `Rider` is independently overridden:
  - In `BikeRider`:  $\text{Time} = \text{Distance} / 30.0 \text{ km/h}$ .
  - In `CycleRider`:  $\text{Time} = \text{Distance} / 12.0 \text{ km/h}$ .

At runtime, the JVM inspects the concrete object reference held by the upcasted `Rider` pointer and dynamically binds the correct calculation routine without requiring explicit type casting.

- **3. Role Description Abstraction:** `getRoleDescription()` is overridden in `BikeRider` to display fuel and distance capacity, and in `CycleRider` to display cargo payload limits.

## 7. Exception Handling

**Relevant class(es)/method(s):** `NoRiderAvailableException`, `RiderAssignmentGUI.createOrderAction()`

- **Custom Checked Exception:** `NoRiderAvailableException` extends `Exception`.
- **Throw Triggers:**
  1. **Fleet Exhaustion:** When all registered riders in the fleet have `isAvailable == false`.
  2. **Ineligible Vehicle Assignment:** When the user requests a `Cycle` preference for a trip exceeding 5.0 km.
  3. **Category Saturation:** When all riders of a specifically requested vehicle preference are occupied.
- **Defensive Recovery Mechanism:** In `RiderAssignmentGUI.createOrderAction()`, dispatching is wrapped in a defensive `try-catch` block. When `NoRiderAvailableException` is caught, the order is retained as "Pending" on the active order board, tables are refreshed, and an informative dialog alerts the dispatcher without crashing the application.
- **Input Validation:** `NumberFormatException` is caught when parsing numerical input fields (distance, fuel level, max load), protecting the application against non-numeric input errors.

## 8. Collections

**Relevant class(es)/method(s):** `DeliveryManager`: `riderMap` (`HashMap`), `orderList` (`ArrayList`), `processPendingOrders()`

- **`HashMap<String, Rider> riderMap`:** Chosen for expected  $O(1)$  performance during frequent rider state lookups and availability mutations. Populated via `riderMap.put(id, rider)` and queried via `riderMap.get(id)`.
- **`ArrayList<Order> orderList`:** Chosen to preserve chronological order sequence and manage the pending queue. Populated via `orderList.add(order)`, traversed in FIFO order during `processPendingOrders()`, and cleared of delivered orders via `orderList.removeIf(...)`.

## 9. Graphical User Interface (GUI)

**Relevant class(es)/method(s):** `RiderAssignmentGUI`: `registerRiderAction()`, `createOrderAction()`, `completeDeliveryAction()`, `deleteSelectedOrderAction()`

Constructed in Java Swing adhering to a responsive dual-panel dashboard architecture:

- **`btnAddRider` -> `registerRiderAction()`:** Reads form inputs, instantiates `BikeRider` or `CycleRider`, updates `DeliveryManager`, triggers pending order execution, and refreshes tables.
- **`btnAssignOrder` -> `createOrderAction()`:** Creates an `Order`, evaluates trip distance and preferences, executes dynamic method overriding for estimated duration, catches custom exceptions gracefully, and resets input fields via `clearOrderForm()`.
- **`btnCompleteOrder` -> `completeDeliveryAction()`:** Identifies the selected active rider, releases the rider, marks the order as "Delivered", and triggers `processPendingOrders()` to reallocate idle couriers.
- **`btnDeleteDelivered` -> `deleteSelectedOrderAction()`:** Verifies that the selected order status equals "Delivered", confirms via prompt, and removes the record permanently from memory.

## 10. Program Output

**Console Output Screenshot(s):**

```
=== INITIALIZING FLEET (2 BIKE RIDERS, 2 CYCLE RIDERS) ===
Registered: Alice (Bike)      | Fuel: 85.0% | Max Dist: 42.5 km | Available: true
Registered: Bob (Bike)       | Fuel: 70.0% | Max Dist: 35.0 km | Available: true
Registered: Joshef (Cycle)   | Max Load: 12.0 kg | Available: true
Registered: Kelvin (Cycle)  | Max Load: 10.0 kg | Available: true

=== TESTING POLYMORPHIC TRAVEL TIME ESTIMATION ===
Trip Distance: 15.0 km
Bike Estimated Duration: 0.50 hours (30.0 km/h)
Cycle Estimated Duration: 1.25 hours (12.0 km/h)

=== TESTING SMART DISPATCH & EXCEPTION RECOVERY ===
Dispatching Order ORD-01 (Dist: 3.5 km, Mode: Any) -> Assigned to Joshef (Cycle)
Dispatching Order ORD-02 (Dist: 4.0 km, Mode: Any) -> Assigned to Kelvin (Cycle)
Dispatching Order ORD-03 (Dist: 15.0 km, Mode: Any) -> Assigned to Alice (Bike)
Dispatching Order ORD-04 (Dist: 8.0 km, Mode: Any) -> Assigned to Bob (Bike)
Dispatching Order ORD-05 (Dist: 6.0 km, Mode: Any) ->
```

[CAUGHT EXPECTED EXCEPTION]: No suitable riders available to deliver Order #ORD-05 (6.0 km)  
Order ORD-05 safely retained as 'Pending'. System stable.

### GUI Screenshot(s):

[Note: Insert desktop GUI screenshots here illustrating: (1) System startup with 4 pre-seeded riders, (2) Calculated delivery time dialog for assigned orders, (3) Peak rush-hour NoRiderAvailableException error dialog, and (4) Live table showing order completion and deletion.]

## 11. Knowledge Profile (KP) Reflection

| #  | Knowledge Area                   | Definition (BAETE) & Guiding Question                                                                                                                                                                                                                                                                         | Your Response                                                                                                                                                                                                                                                                                                                                                                                    |
|----|----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| K1 | Natural Sciences                 | <p>Theory-based understanding of the natural sciences relevant to the discipline.</p> <p>Often Not Applicable for a software project — state clearly if no natural-science principles apply, or note if your scenario models a physical/scientific quantity (e.g. a sensor reading, a measured capacity).</p> | <p>Applies? <input type="checkbox"/> Yes <input checked="" type="checkbox"/> Partially <input type="checkbox"/> No</p> <p><b>How:</b> Models kinematic physical transport principles: vehicle speed, travel time (<math>t = d/v</math>), fuel percentage depletion constraints for combustion engines, and payload weight limits for bicycles.</p>                                               |
| K2 | Mathematics                      | <p>Mathematics, numerical analysis, statistics, and formal computer/information science used for analysis and modelling.</p> <p>e.g. numeric computations, comparisons, or thresholds in your class methods (a running total, an average, a capacity limit that triggers your custom exception).</p>          | <p>Applies? <input checked="" type="checkbox"/> Yes <input type="checkbox"/> Partially <input type="checkbox"/> No</p> <p><b>How:</b> Implements kinematic duration equations, distance-threshold evaluation formulas (<math>\leq 5.0</math> km vs. <math>&gt; 5.0</math> km), range mapping functions, and <math>O(1)</math> computational complexity analysis for hash lookups.</p>            |
| K3 | Engineering Fundamentals         | <p>Theory-based engineering fundamentals underlying the discipline — for CSE, core computing/software principles.</p> <p>e.g. the core OOP fundamentals your whole design rests on: classes, objects, encapsulation, and control-flow logic.</p>                                                              | <p>Applies? <input checked="" type="checkbox"/> Yes <input type="checkbox"/> Partially <input type="checkbox"/> No</p> <p><b>How:</b> Architected entirely upon core software principles: data encapsulation, information hiding via access specifiers, reference passing, constructor chaining, and defensive control-flow logic.</p>                                                           |
| K4 | Specialist Engineering Knowledge | <p>Discipline-specific specialist knowledge and frameworks, including current practice areas — e.g. object-oriented software design.</p> <p>e.g. the specific techniques this course covers: multilevel inheritance, polymorphism, interfaces, exception handling, and the Collections Framework.</p>         | <p>Applies? <input checked="" type="checkbox"/> Yes <input type="checkbox"/> Partially <input type="checkbox"/> No</p> <p><b>How:</b> Synthesizes core OOP paradigms: 3-level multilevel inheritance, dynamic runtime method binding, interface realization, checked exception propagation, and the Java Collections Framework.</p>                                                              |
| K5 | Engineering Design               | <p>Knowledge that supports design in the practice area — e.g. class/system design, design patterns.</p> <p>e.g. how you designed your class hierarchy, chose which class owns which responsibility, or laid out your GUI.</p>                                                                                 | <p>Applies? <input checked="" type="checkbox"/> Yes <input type="checkbox"/> Partially <input type="checkbox"/> No</p> <p><b>How:</b> Employs Model-View-Controller (MVC) architectural separation, isolating domain entities (<code>Rider</code>, <code>Order</code>), business controllers (<code>DeliveryManager</code>), and Swing presentation views (<code>RiderAssignmentGUI</code>).</p> |

| #  | Knowledge Area          | Definition (BAETE) & Guiding Question                                                                                                                                                                                                                                                              | Your Response                                                                                                                                                                                                                                                                                                                                   |
|----|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| K6 | Engineering Practice    | <p>Knowledge of accepted engineering practice/technology in the field — e.g. IDEs, coding standards, testing practices.</p> <p>e.g. your use of a Java IDE, version control, naming conventions, code comments, or a testing routine you followed.</p>                                             | <p>Applies? <input checked="" type="checkbox"/> Yes <input type="checkbox"/> Partially <input type="checkbox"/> No</p> <p><b>How:</b> Developed in Visual Studio Code with JDK 17, adhering to standard Java naming conventions (camelCase), Git version control, defensive input validation, and iterative visual testing routines.</p>        |
| K7 | Engineering and Society | <p>Understanding of engineering's role in society: ethics, professional responsibility, and social/economic/environmental impact.</p> <p>e.g. how your exception handling or input validation protects a user from unsafe use, or why your scenario's real-world context matters to its users.</p> | <p>Applies? <input checked="" type="checkbox"/> Yes <input type="checkbox"/> Partially <input type="checkbox"/> No</p> <p><b>How:</b> Implements an eco-routing framework prioritizing human-powered bicycles for short trips (<math>\leq 5</math> km) to reduce urban carbon emissions, while preventing commercial double-booking errors.</p> |
| K8 | Research Literature     | <p>Engagement with research literature or authoritative technical references in the discipline.</p> <p>e.g. the Java API documentation, Oracle's official tutorials, or any other technical reference you consulted beyond the lecture notes.</p>                                                  | <p>Applies? <input checked="" type="checkbox"/> Yes <input type="checkbox"/> Partially <input type="checkbox"/> No</p> <p><b>How:</b> Consulted official Oracle Java SE Documentation (Collections Framework specifications, Swing Event Dispatch Thread rules) and transport logistics literature on dynamic vehicle dispatch.</p>             |

## 12. Complex Engineering Problem (CEP) Reflection

| #  | Attribute                         | Definition (BAETE) & Guiding Question                                                                                                                                                                                                                                                                                                                                           | Your Response                                                                                                                                                                                                                                                                                                                                                                                          |
|----|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| P1 | Depth of Knowledge Required       | <p>Resolved using in-depth, fundamentals-based engineering/specialist knowledge — not solved by routine or superficial application of a single taught example.</p> <p>e.g. does your design require you to genuinely apply OOP fundamentals — inheritance, polymorphism, exception design, the Collections Framework — in an integrated way, rather than each in isolation?</p> | <p>Addressed? <input checked="" type="checkbox"/> Yes <input type="checkbox"/> Partially <input type="checkbox"/> Not Applicable</p> <p><b>Justification:</b> Requires deep, integrated OOP synthesis: abstract class inheritance, runtime method overriding, interface decoupling, checked exception handling, and hash collections working cohesively.</p>                                           |
| P2 | Range of Conflicting Requirements | <p>Involves wide-ranging or conflicting technical and other issues that must be balanced against each other.</p> <p>e.g. does your scenario force trade-offs — simplicity vs. robustness, a flexible class hierarchy vs. a simple one, GUI usability vs. development time?</p>                                                                                                  | <p>Addressed? <input checked="" type="checkbox"/> Yes <input type="checkbox"/> Partially <input type="checkbox"/> Not Applicable</p> <p><b>Justification:</b> Balanced urban eco-efficiency (preferring bicycles) against delivery latency (bicycles are slower) and range constraints, alongside balancing GUI real-time responsiveness against collection updates.</p>                               |
| P3 | Depth of Analysis Required        | <p>Has no single obvious solution; requires abstract thinking and originality to formulate a suitable design.</p> <p>e.g. did you have to weigh multiple possible class hierarchies, collection types, or exception designs before settling on one, rather than there being one “correct” answer?</p>                                                                           | <p>Addressed? <input checked="" type="checkbox"/> Yes <input type="checkbox"/> Partially <input type="checkbox"/> Not Applicable</p> <p><b>Justification:</b> Evaluated multiple data structure alternatives (ArrayList vs. HashMap vs. PriorityQueue) and formulated specialized vehicle modeling without duplicating common identity data.</p>                                                       |
| P4 | Familiarity of Issues             | <p>Involves issues that are infrequently encountered, rather than routine, well-rehearsed problems.</p> <p>e.g. does your custom exception model an edge case or failure condition that is not simply the standard textbook example?</p>                                                                                                                                        | <p>Addressed? <input checked="" type="checkbox"/> Yes <input type="checkbox"/> Partially <input type="checkbox"/> Not Applicable</p> <p><b>Justification:</b> Fleet exhaustion is a critical real-world edge case. Designing a custom checked exception to halt dispatch gracefully and queue orders rather than throwing standard null-pointer runtime errors resolved this non-trivial scenario.</p> |

| #  | Attribute                         | Definition (BAETE) & Guiding Question                                                                                                                                                                                                                                                                                       | Your Response                                                                                                                                                                                                                                                                                                                                                                                |
|----|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| P5 | Extent of Applicable Codes        | <p>Goes beyond standard codes of practice — typically most relevant to safety-critical or regulated engineering domains.</p> <p>Often Not Applicable at this project scale — state clearly if no formal codes of practice apply to your scenario, or note any coding/security standard you chose to follow voluntarily.</p> | <p>Addressed? <input type="checkbox"/> Yes <input checked="" type="checkbox"/> Partially <input type="checkbox"/> Not Applicable</p> <p><b>Justification:</b> Enforced defensive programming validation standards across all input boundaries and constructed GUI components strictly adhering to Java Swing threading rules.</p>                                                            |
| P6 | Extent of Stakeholder Involvement | <p>Involves diverse groups of stakeholders with widely varying needs and requirements.</p> <p>e.g. does your GUI need to serve a non-technical end user with different needs than an administrator, or does your scenario involve more than one type of user role?</p>                                                      | <p>Addressed? <input checked="" type="checkbox"/> Yes <input type="checkbox"/> Partially <input type="checkbox"/> Not Applicable</p> <p><b>Justification:</b> Balances competing requirements of three stakeholder groups: dispatchers (operational visibility and fast lookups), riders (fair allocation and realistic travel limits), and customers (rapid delivery).</p>                  |
| P7 | Interdependence                   | <p>Is a high-level problem made up of many interdependent component parts or sub-problems.</p> <p>e.g. do your class hierarchy, exception handling, collections, and GUI all have to work together correctly for the system to function — a change in one affecting the others?</p>                                         | <p>Addressed? <input checked="" type="checkbox"/> Yes <input type="checkbox"/> Partially <input type="checkbox"/> Not Applicable</p> <p><b>Justification:</b> High architectural interdependence: modifications in the <code>Rider</code> inheritance tree directly impact polymorphic calls in <code>DeliveryManager</code>, altering exception signatures and GUI event handler logic.</p> |

## 13. Complex Engineering Activity (CEA) Reflection

| #  | Attribute                                    | Definition (BAETE) & Guiding Question                                                                                                                                                                                                                                         | Your Response                                                                                                                                                                                                                                                                                                                                                                        |
|----|----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| A1 | Range of Resources                           | <p>Involves the use of diverse resources — people, time, information, and technologies.</p> <p>e.g. your IDE, the Java API documentation, Swing/AWT references, and any instructor consultation or peer discussion used during development.</p>                               | <p>Addressed? <input checked="" type="checkbox"/> Yes <input type="checkbox"/> Partially <input type="checkbox"/> Not Applicable</p> <p><b>Justification:</b> Leveraged Java SE SDK, Visual Studio Code IDE, Java Swing libraries, Git version control, lab instructor consultations, and collaborative peer-programming sessions.</p>                                               |
| A2 | Level of Interaction                         | <p>Requires resolving significant problems arising from interactions between wide-ranging or conflicting technical issues.</p> <p>e.g. resolving a conflict between how your inheritance hierarchy is structured and how the GUI needs to display or update object state.</p> | <p>Addressed? <input checked="" type="checkbox"/> Yes <input type="checkbox"/> Partially <input type="checkbox"/> Not Applicable</p> <p><b>Justification:</b> Resolved synchronization between <code>DefaultTableModel</code> UI table updates and <code>DeliveryManager</code> collection state changes during the automatic reassignment of pending orders upon rider release.</p> |
| A3 | Innovation                                   | <p>Involves creative use of engineering principles and knowledge in novel ways.</p> <p>e.g. the originality of your chosen scenario, or a design choice (collection strategy, exception design) you arrived at rather than copied from a lab example.</p>                     | <p>Addressed? <input checked="" type="checkbox"/> Yes <input type="checkbox"/> Partially <input type="checkbox"/> Not Applicable</p> <p><b>Justification:</b> Developed an autonomous pending order reallocation pipeline (<code>processPendingOrders</code>) that triggers upon new rider registration and delivery completion, emulating commercial event buses.</p>               |
| A4 | Consequences for Society and the Environment | <p>Has significant consequences in a range of contexts, where predicting and mitigating those consequences is difficult.</p>                                                                                                                                                  | <p>Addressed? <input checked="" type="checkbox"/> Yes <input type="checkbox"/> Partially <input type="checkbox"/> Not Applicable</p> <p><b>Justification:</b> Promotes environmental sustainability by prioritizing non-motorized bicycle couriers for short-range</p>                                                                                                               |

| #  | Attribute   | Definition (BAETE) & Guiding Question                                                                                                                                                                                                                                                 | Your Response                                                                                                                                                                                                                                                                                                                                             |
|----|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|    |             | Often limited at this project scale — state clearly if your scenario has minimal real-world consequence, or note any data-safety/usability consideration that does apply (e.g. safe handling of invalid input).                                                                       | logistics, directly modeling emissions reduction in municipal delivery ecosystems.                                                                                                                                                                                                                                                                        |
| A5 | Familiarity | <p>Extends beyond previous experience, requiring you to apply principles-based approaches to a new or unfamiliar situation.</p> <p>e.g. does your scenario or a specific technique (e.g. a Swing component, a collection type) go beyond what was directly demonstrated in class?</p> | <p>Addressed? <input checked="" type="checkbox"/> Yes <input type="checkbox"/> Partially <input type="checkbox"/> Not Applicable</p> <p><b>Justification:</b> Extended beyond simple console examples by architecting an end-to-end full-stack desktop software solution with graphical event dispatching, custom exceptions, and nested collections.</p> |

## 14. Challenges Encountered and How They Were Resolved

| Problem Encountered                                                                                                                                                                                                   | How You Resolved It                                                                                                                                                                                                                   |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>1. Truncated Table Details Column:</b> The descriptive string returned by <code>getRoleDescription()</code> was clipped with ellipses ( . . . ) in the Swing table due to small default cell widths.               | Configured table auto-resize behavior to <code>JTable.AUTO_RESIZE_OFF</code> and explicitly set the preferred width of Column 4 (Details) to 320 pixels within a horizontal scroll pane.                                              |
| <b>2. Unrealistic Long-Distance Bicycle Dispatches:</b> In early iterations, a 25 km delivery order was assigned to a bicycle courier because the system naively grabbed the first available rider in the collection. | Restructured <code>assignRider()</code> to evaluate trip distance: distances > 5.0 km strictly query and allocate <code>BikeRider</code> instances; bicycle dispatches > 5.0 km trigger an explicit exception.                        |
| <b>3. Sticky Input Fields Post-Assignment:</b> Order details remained populated in input text boxes after dispatch, requiring tedious manual backspacing for every subsequent customer order.                         | Created an isolated helper routine <code>clearOrderForm()</code> that resets all text fields and radio button groups, invoked immediately upon order creation and exception recovery.                                                 |
| <b>4. Orphaned Pending Orders:</b> When an order entered "Pending" status during peak hours, it remained stranded indefinitely even after busy riders completed deliveries and became available.                      | Implemented an event-driven <code>processPendingOrders()</code> loop invoked automatically inside <code>completeDeliveryAction()</code> and <code>registerRiderAction()</code> , seamlessly matching idle riders with waiting orders. |

## 15. Conclusion

This project successfully engineered and demonstrated the **Smart Food Delivery Rider Assignment & Dispatch System**, addressing real-world logistical bottlenecks in urban food delivery operations. Through the rigorous application of Object-Oriented Programming in Java, the system proves that abstract class hierarchies, interface contracts, and runtime polymorphism can elegantly model complex, real-world physical workflows. By integrating distance-tiered dispatch algorithms with custom checked exceptions, the software delivers resilient performance under peak-load saturation without unhandled failures. The implementation of Java Swing and Collections Framework components highlights the vital connection between theoretical software engineering principles and production-grade desktop application development.

## Declaration

I declare that this report and the accompanying source code / configuration (if any) are my own original work, prepared for this course, and that all external references or assistance used have been acknowledged.

|                                          |                              |
|------------------------------------------|------------------------------|
| Student Signature: <i>Pranta Mohajan</i> | Student ID: 0222510005101025 |
|------------------------------------------|------------------------------|

|                          |  |
|--------------------------|--|
| Date: September 29, 2026 |  |
|--------------------------|--|